Practical No 09

A) Develop CRUD operations using ASP.NET Core Web API and Entity Framework core.

Project Name: StudentAPIWithEF
Project Type: ASP.NET Core Web API
Program Code:

I. Filename: Student.cs

```
namespace StudentAPIWithEF.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
        public DateTime CreatedDate { get; set; }
    }
}
```

II. Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }
        public DbSet<Student> Students { get; set; }
    }
}
```

III. Filename: StudentsController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentAPIWithEF.Data;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public StudentsController(AppDbContext context)
        {
            _context = context;
        }

        // GET ALL
        [HttpGet]
        public IActionResult GetStudents()
        {
            return Ok(_context.Students.ToList());
        }

        // GET BY ID
        [HttpGet("{id}")]
        public IActionResult GetStudent(int id)
        {
            var student = _context.Students.Find(id);

            if (student == null)
                return NotFound();

            return Ok(student);
        }

        // INSERT
        [HttpPost]
        public IActionResult AddStudent(Student student)
        {
            _context.Students.Add(student);
            _context.SaveChanges();

            return Ok(student);
        }

        // UPDATE
        [HttpPut("{id}")]
        public IActionResult UpdateStudent(int id, Student student)
        {
            var data = _context.Students.Find(id);

            if (data == null)
                return NotFound();

            data.Name = student.Name;
            data.Age = student.Age;
            data.Course = student.Course;

            _context.SaveChanges();

            return Ok(data);
        }

        // DELETE
        [HttpDelete("{id}")]
        public IActionResult DeleteStudent(int id)
```

```
        {
            var student = _context.Students.Find(id);

            if (student == null)
                return NotFound();

            _context.Students.Remove(student);
            _context.SaveChanges();

            return Ok();
        }
    }
}
```

IV. Filename: appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

V. Filename: Program.cs

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

VI. Filename: launchSettings.json

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5293"
    },
    "https": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "https://localhost:7103;http://localhost:5293"
    }
  }
}
```

VII. Package Manager Console commands

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools

Add-Migration InitialCreate
Update-Database
```

B) Create an API endpoint to calculate Factorial of a Number and test it using Postman.

Project Name: FactorialAPI
Project Type: ASP.NET Core Web API
Program Code:

I. Filename: MathController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult Factorial(int number)
        {
            if (number < 0)
            {
                return BadRequest(new
                {
                    message = "Factorial is not defined for negative numbers."
                });
            }

            if (number > 20)
            {
                return BadRequest(new
                {
                    message = "Number too large. Enter a value between 0 and 20."
                });
            }

            long factorial = 1;

            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }

            return Ok(new
            {
                number = number,
                factorial = factorial
            });
        }
    }
}
```

II. Filename: launchSettings.json

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5293"
    },
    "https": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "https://localhost:7103;http://localhost:5293"
    }
  }
}
```

III. Testing using Postman

```
Method : GET
URL    : https://localhost:7103/api/math/factorial/5

Expected response (200 OK):
{
    "number": 5,
    "factorial": 120
}

Negative number: GET https://localhost:7103/api/math/factorial/-3
Response (400 Bad Request):
{
    "message": "Factorial is not defined for negative numbers."
}
```